StudyForge AI

Complete Project Study Guide for PPT and Viva

A detailed, easy-language guide based on the actual current codebase.

For BTech project presentation and viva

How to Use This Guide

Study this guide in two passes. In the first pass, understand the student story and architecture. In the second pass, open the mentioned files and follow the code. You do not need to memorise every line. In a viva, explain responsibility, data flow and reason for each decision.

Goal	Read first	Outcome
PPT preparation		
	1 to 5 and 14	
		Explain problem, solution, features and architecture.
Code demonstration		
	6 to 13	
		Show which file implements a feature.
Viva preparation		
	15 to 18	
		Give accurate answers, limits and future scope.

Learning Outcomes

- Explain StudyForge AI in one minute.
- Draw React to FastAPI to SQLite to AI architecture.
- Explain PDF upload, chunking, retrieval, tokens, quiz logic and fallbacks.
- Show a reliable live demo.
- Answer technical viva questions honestly.

Contents

1. Project in Simple Words
2. Technology and folder map
3. Architecture and data flow
4. Backend foundations
5. Database and authentication
6. PDF upload and RAG
7. AI chat and streaming
8. Flashcards, quizzes and fallback
9. Frontend and design
10. Run, test and debug
11. PPT plan
12. Viva questions
13. Code walkthrough plan
14. Limitations and revision checklist

1 Project in Simple Words

StudyForge AI is an AI-powered study workspace for BTech students. A student creates an account, uploads a PDF and uses that same material for source-based chat, flashcards, quizzes, recall practice, revision tracking, tasks and a personal profile. The assistant is named ASHIEY.

One Minute Project Explanation

StudyForge AI turns a student PDF into an organised study workspace. Next.js and React build the frontend. FastAPI handles the backend. SQLite stores accounts, documents, chunks, chats and scores. PyMuPDF extracts text from uploaded PDFs. The backend breaks text into chunks and retrieves relevant chunks using TF IDF and cosine similarity. Gemini can write natural explanations and study material. When the provider is unavailable, local fallbacks still allow the student to study.

Problem Statement and Objectives

Student problem	Project response	Value
Long PDFs are difficult to revise		
	Upload, chat, cards and quizzes	
		Active recall from own material.
Study work is scattered		
	Library, planner, revision tracker	
		One organised workspace.
Generic AI can ignore notes		
	Retrieval from student PDF chunks	
		More grounded answers.
Provider may be busy		
	Local retrieval and quiz fallback	
		Feature does not become a dead end.

- Provide a personalised student workspace.
- Accept PDFs safely with source pages.
- Use relevant uploaded material for answers.
- Generate distinct study questions.
- Track planning, revision and quiz progress.

2 Technology Stack

Layer	Technology	Reason	Where
Frontend			
	Next.js 15, React 19, TypeScript		
		Component UI and type checking	
			frontend/app
Styling			

	CSS and React state		
		Custom warm visual identity	
			frontend/app CSS
Backend			
	FastAPI and Uvicorn		
		Validated Python API	
			backend/app/main.py
Database			
	SQLite		
		No separate local DB server	
			backend/data runtime
PDF			
	PyMuPDF		
		Extract text page by page	
			upload_document
Retrieval			
	scikit-learn TF IDF and cosine similarity		
		Fast local search	
			retrieve
AI			
	Google GenAI Gemini and optional Groq		
		Natural text generation	
			ask_gemini and stream_groq
Testing			
	pytest and Playwright scripts		
		Checks feature behaviour	
			tests and check files

Important viva statement: the current project uses TF IDF retrieval. It does not use a vector database or semantic embeddings. Saying this accurately is stronger than claiming unimplemented technology.

3 Folder Structure and File Map

```
STUDYFORGE_AI/
backend/
  app/main.py                FastAPI routes and core logic
  app/study.py               Library, planner, revision and saved chat routes
  tests/                     pytest tests
  requirements.txt           Python packages
frontend/
  app/page.tsx               Main React screen and navigation
  app/ashiey-chat.tsx        Chat interface
  app/library.tsx            Subject and PDF library
  app/planner.tsx            Planner interface
  app/profile-setup.tsx      Student profile
  app/avatar-creator-3d.tsx  Avatar creator
  app/*.css                  Theme and visual system
  package.json               JavaScript dependencies
START-STUDYFORGE.cmd         Windows launcher
start-studyforge.ps1         Starts both services
README.md                    Project instructions
STUDY-FEATURES.md            Feature behaviour and limits
```

The frontend is responsible for the browser interface. The backend is responsible for validation, business logic, database access, PDF processing and provider calls. The database, uploads, environment files and installed packages are local runtime material and are not committed to Git.

4 End to End Architecture

```
Student browser
   ↓ Bearer token plus JSON or PDF form data
Next.js React frontend on port 3000
   ↓
FastAPI backend on port 8000
   ├─ SQLite: users, documents, chunks, chats, quizzes, tasks
   ├─ PyMuPDF: PDF text extraction
   ├─ TF IDF retrieval: relevant chunks
   └─ Gemini or Groq: natural generation when configured
   ↓ JSON or newline-delimited streaming
React updates the study workspace
```

Typical PDF Question Flow

15. Student signs in and frontend saves the signed token in local storage.
16. Student uploads PDF through POST /api/documents/upload.
17. FastAPI validates user and PDF, then PyMuPDF extracts page text.
18. split_text makes overlapping chunks and SQLite saves text with page number.
19. retrieve ranks chunks using TF IDF and cosine similarity.
20. Gemini can receive relevant context and the document to write an answer.
21. Frontend shows answer and source information; chat may be saved.

Boundary	Role	Why it matters
React and FastAPI		
	Interface versus trusted server logic	
		Browser cannot directly access database or key.
FastAPI and provider		

	Application logic versus external service	
		Validation and fallback remain under project control.
SQLite and original PDF		
	Structured records versus source file	
		Fast querying and retained original material.
Retrieval and generation		
	Find evidence versus compose answer	
		More focused answers and source pages.

5 Backend Foundations

backend/app/main.py contains the core FastAPI app. Decorators such as @app.post and @app.get connect a URL and HTTP method to a function. Pydantic models validate data before application logic starts.

```
@app.post("/api/quizzes/generate")
def generate_quiz(request: GenerateRequest, user=Depends(current_user)):
    owned_document(request.document_id, user)
    # create questions only for the logged in student's PDF
    ...
```

GenerateRequest validates document ID and count. Depends(current_user) asks FastAPI to authenticate first. owned_document confirms that the selected PDF belongs to the logged-in student. This is an authorization check.

Model	Purpose	Fields
SignupRequest and LoginRequest		
	Validate account data	
		name, email, password
PreferencesRequest and ProfileRequest		
	Save profile	
		subjects, goal, branch, semester
ChatRequest and ChatMessage		
	Describe chat input and history	
		question, document ID, mode
GenerateRequest		
	Control generation	
		document ID, count from 1 to 20
QuizSubmission		
	Receive selected answers	
		answers by question ID
AvatarRequest		
	Save appearance and gender	
		avatar data, gender

HTTP Status Codes

Code	Meaning	Example
200		
	Success	
		Profile, document list, quiz result
400		
	Invalid request	
		Non-PDF or unreadable PDF
401		
	Invalid login	

		No Bearer token
404		
	Missing or not owned item	
		Wrong document ID
413		
	Upload too large	
		PDF above 20 MB
422		
	Cannot form valid result	
		Too few distinct quiz questions
502		
	Bad provider response	
		Malformed AI JSON
503		
	Provider unavailable	
		AI busy

6 Database and Authentication

SQLite is an embedded database stored in one file. It is suitable for a local BTech project because setup is simple. db_connection returns sqlite3.Row objects so code reads values using column names.

```
def db_connection() -> sqlite3.Connection:
    db = sqlite3.connect(DATABASE_PATH)
    db.row_factory = sqlite3.Row
    return db
```

Table	Stores	Relation
users		
	account, preferences, avatar	
		Owns student records
documents		
	PDF metadata and owner	
		Has chunks, cards and quizzes
chunks		
	page and extracted text	
		Used by retrieval
conversations		
	saved chat messages	
		Allows chat continuation
subjects and document_subjects		
	library organisation	
		Groups materials

student_tasks and revision_topics		
	planning and revision status	
		Owned by user
quizzes and quiz_attempts		
	question set and result	
		Attempts linked to user
flashcards		
	question answer pairs	
		Linked to document

Password and Token Flow

22. password_hash creates a random salt.
23. PBKDF2 HMAC SHA 256 runs 120000 iterations; raw passwords are never stored.
24. password_matches recomputes with the stored salt.
25. create_token makes a user ID and seven-day expiry payload signed with HMAC SHA 256.
26. Frontend sends Authorization Bearer token on protected routes.
27. current_user validates signature and expiry then loads the user.

Precise viva answer: this is a custom base64 and HMAC signed token, not a JWT library implementation. A production version should use standard session or JWT tooling with key rotation.

7 PDF Upload and RAG

Rule	Implementation	Reason
PDF only		
	Content type must be application/pdf	
		Rejects unsupported files early
Size		
	Maximum 20 MB	
		Bounds local resource use
Pages		
	1 to 200	
		Keeps processing predictable
Encryption		
	pdf.needs_pass check	
		Cannot reliably extract protected text
Ownership		
	Document stores user ID	
		Protects student material

Chunking

A full PDF is too broad to search as one item. split_text creates chunks of 180 words with 30 words of overlap. Overlap keeps an idea visible when it lies at the edge of two chunks. Every stored chunk retains its PDF page.

```
def split_text(text: str, size: int = 180, overlap: int = 30):
    words = text.split()
    return [" ".join(words[i:i + size])
            for i in range(0, len(words), size - overlap)]
```

RAG in the Actual Code

RAG means Retrieval Augmented Generation. Retrieval selects useful source material before generation writes the answer. In this project, TfidfVectorizer converts chunks and the question into weighted word vectors. cosine_similarity scores closeness. The top four positive chunks become sources.

```
matrix = TfidfVectorizer(stop_words="english").fit_transform(texts + [question])
scores = cosine_similarity(matrix[-1], matrix[:-1]).flatten()
best = scores.argsort()[::-1][:limit]
```

TF IDF raises the importance of a word that is useful to one chunk but uncommon across all chunks. Cosine similarity compares vector angle. This is fast and local, but it can miss a concept written with entirely different words.

8 AI Chat and Streaming

Function	Responsibility
gemini_client	
	Creates Google GenAI client if key exists.
ask_gemini	

	Produces non-streaming answer.
stream_gemini	
	Yields response pieces.
stream_groq	
	Streams chat with bounded retry.
chat_provider	
	Chooses configured provider.
chat_failure	
	Makes provider error student-friendly.
fallback_answer	
	Returns retrieved source text if AI is unavailable.

The /api/chat/stream endpoint uses newline-delimited JSON. It first emits status, then delta events with small answer pieces, then a done event. Streaming gives immediate feedback for long responses.

```
yield event({"type": "status", "message": "Reading your PDF…"})
for text in chunks:
    answer += text
    yield event({"type": "delta", "text": text})
yield event({"type": "done", **result})
```

9 Flashcards Quizzes and Local Fallback

The study tools are designed to avoid rewording one concept many times. The provider is asked for more candidates than the student requests. Then the backend filters weak or similar items and attaches a short focus label.

Function	Role
study_words	
	Keeps meaningful words and removes filler.
study_text_is_similar	
	Detects near duplicate wording.
focus_from_question	
	Creates a concept label for UI.
keep_distinct_study_items	
	Retains valid and different items.
generate_document_items	
	Requests and validates provider items.
make_local_quiz	
	Builds source-based fallback during provider outage.

Quiz Creation Flow

28. Student chooses a PDF and presses Create quiz.
29. Frontend sends document ID and count.
30. Backend checks ownership and prefers distinct Gemini questions.
31. Backend validates four options, correct answer, explanation, page and focus.
32. If no provider questions are available, make_local_quiz reads distinct source sentences.
33. Complete questions are stored in SQLite. The generation response hides answer and explanation.
34. On submit, backend compares answers, saves attempt and awards 10 XP per correct answer.

Why Local Fallback is Important

A provider can be temporarily busy. A useful study product should not only display a dead-end error. The fallback creates a source-grounded question and uses sentences from other topics as distractors. It is more literal than Gemini but keeps the feature available.

```
questions = keep_distinct_study_items(
    ask_gemini_quiz(request.document_id, request.count) or [], request.count, quiz=True)
if not questions:
    questions = make_local_quiz(request.document_id, request.count)
```

10 Frontend and User Experience

frontend/app/layout.tsx loads CSS, KaTeX styles, ThemeToggle and MotionControl. frontend/app/page.tsx is the main client component. It holds login state, page selection, selected document, notifications, flashcards, quiz and progress.

State	Meaning
token and user	

	Shows login or workspace
page	
	hub, planner, library, revision, chat, cards, quiz, games or progress
documentId and documentName	
	Selected study PDF
busy and notice	
	Button state and feedback
cards, quiz and quizId	
	Generated practice material
chatStart	
	Initial question, mode and document for chat

```
const response = await fetch(API + "/api/" + kind + "/generate", {
  method: "POST",
  headers: { "Content-Type": "application/json", Authorization: "Bearer " + token },
  body: JSON.stringify({ document_id: documentId, count: 5 })
})
const data = await response.json()
if (!response.ok) throw new Error(data.detail || "Could not generate study material.")
```

This pattern sends JSON, includes the signed token, reads JSON and turns unsuccessful responses into visible friendly feedback. The API URL is configurable through NEXT_PUBLIC_API_URL and defaults to local port 8000.

11 Avatar Theme and Accessibility

avatar-creator-3d.tsx and student-avatar-3d.tsx provide avatar customisation. PUT /api/profile/avatar saves avatar configuration and gender selection. The backend stores this data as JSON in the users table.

Say this in viva: the avatar has a 3D-styled appearance created with layered React and CSS. It avoids a heavy WebGL engine while still providing depth and personalisation. Do not claim Three.js or WebGL because the project does not use them.

Area	Implementation	Benefit
Visual identity		
	Warm ivory, lavender and ink styling	
		Student-focused and handmade feel
Theme		
	ThemeToggle plus local storage	
		Light and dark comfort
Motion		
	MotionControl and reduced-motion support	
		Does not force animation
Chat rendering		
	Text rather than trusted raw HTML	
		Reduces script injection risk
Code blocks		
	Copy button and manual fallback	
		Works if clipboard is blocked
Ownership		
	Protected routes validate account and document	
		Avoids cross-account access

12 Running Testing and Debugging

```
cd "C:\Users\akshita sharda\Documents\ChatGPT\STUDYFORGE_AI"
.\START-STUDYFORGE.cmd
```

The launcher starts Uvicorn with reload on port 8000 and Next development server on port 3000. It waits for both health URLs. STOP-STUDYFORGE.cmd closes services on those ports.

Task	Command
Install backend	
	py -m venv .venv then .venv\Scripts\python.exe -m pip install -r requirements.txt
Run backend	
	.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000

Test backend	
	.venv\Scripts\python.exe -m pytest -q
Install frontend	
	npm.cmd install
Run frontend	
	npm.cmd run dev -- --port 3000
Build frontend	
	npm.cmd run build

Problem	Likely cause	First check
Port already used		
	Previous app instance	
		Open /health or run stop launcher
Provider busy		
	External service issue	
		Retry; local fallback needs readable PDF text
No quiz items		
	Few distinct readable statements	
		Use clearer text PDF
Upload fails		
	Wrong type, size, pages or encryption	
		Check upload rules
Login fails		
	Bad credentials or expired token	
		Sign in and inspect response
Frontend cannot call backend		
	Backend stopped or wrong URL	
		Open 127.0.0.1:8000/health

13 PPT Structure and Demo Script

Use 10 to 12 slides. Keep slides visual and speak the details. Do not put long code on a slide. Use an architecture diagram and only small code snippets.

Slide	Title	What to say
1		
	StudyForge AI	
		Title and one-line purpose
2		
	Problem Statement	
		PDF study and revision are fragmented
3		
	Our Solution	
		Personalised workspace using uploaded material
4		
	Features	
		Upload, chat, quiz, cards, planner, revision and profile
5		
	Architecture	
		React, FastAPI, SQLite, PDF, retrieval and AI
6		
	PDF and RAG	
		Extract, chunk, retrieve TF IDF, generate with sources
7		
	Backend	
		Validation, token, ownership, database
8		
	Frontend	
		Student workspace, theme and avatar
9		
	Reliability	
		Distinct questions and local provider fallback
10		
	Testing	
		pytest, build, browser checks, health
11		
	Limits	
		OCR, embeddings, PostgreSQL and

		security
12		
	Thank You	
		Invite questions and demonstrate

Live Demo Order

35. Sign in and show workspace.
36. Show profile or avatar.
37. Upload a short readable PDF.
38. Ask ASHIEY one question and show source page.
39. Reveal a flashcard.
40. Create, answer and submit a quiz.
41. Show score or progress.
42. Open planner or revision tracker.

Opening Script

Good morning. Our project is StudyForge AI, a study workspace designed for BTech students. Instead of treating PDFs, doubts, revision and planning as separate activities, we bring them into one application. The student uploads study material, the backend extracts and retrieves relevant text, and the frontend turns it into chat, flashcards, quizzes, revision tracking and planning.

Closing Script

Our main engineering decision was to keep student material central. We use local PDF extraction and retrieval before generation, validate backend requests, protect student-owned resources and provide local fallback behaviour when an AI provider is unavailable. This makes the project more useful and reliable than a generic chatbot.

14 Viva Questions and Model Answers

Q. What is StudyForge AI? A. A BTech study workspace using student PDFs for chat, flashcards, quizzes, recall, planning, revision and profile personalisation.

Q. Why FastAPI? A. It provides route decorators, Pydantic validation, dependency injection and automatic API documentation.

Q. Why SQLite? A. It is simple for a local prototype and needs no database server. A deployment could use PostgreSQL.

Q. What is RAG? A. Retrieval Augmented Generation finds relevant source passages before a model writes an answer.

Q. How does retrieval work? A. TF IDF and cosine similarity rank saved PDF chunks against the student question.

Q. What is TF IDF? A. A weighting method that gives importance to words useful in one text but uncommon across all texts.

Q. What is cosine similarity? A. A vector comparison that measures closeness by angle.

Q. Why chunk PDFs? A. Smaller page-linked chunks are easier to retrieve than an entire document, and overlap protects boundary ideas.

Q. How do you validate PDFs? A. Check account, PDF type, 20 MB size, 1 to 200 pages and encryption.

Q. How do you protect documents? A. Each document has user ID and protected routes call current_user plus owned_document.

Q. How are passwords stored? A. PBKDF2 HMAC SHA 256 with random salt and 120000 iterations; never raw passwords.

Q. Is your token JWT? A. No. It is a custom base64 HMAC signed token with expiry. Standard sessions or JWT are future production work.

Q. What happens if Gemini is busy? A. Chat can use retrieved material and quizzes use a local source-grounded fallback.

Q. How do you prevent repetitive quizzes? A. Generate extra candidates and filter paraphrases using meaningful word overlap and sequence similarity.

Q. Why hide answers until submit? A. Complete questions stay on server; generation response removes answer and explanation fields.

Q. Why streaming? A. It shows progress and text as it arrives rather than making the student wait for a full answer.

Q. What is Pydantic? A. FastAPI uses it to parse and validate structured request data.

Q. What is dependency injection? A. Depends(current_user) runs authentication before a route and supplies the verified user.

Q. Is avatar WebGL? A. No. It is a 3D-styled React and CSS component, not a WebGL engine.

Q. What are limitations? A. Poor scans, lexical TF IDF, provider availability and SQLite local scale.

Q. Future improvements? A. OCR, embeddings, vector store, PostgreSQL, standard sessions, rate limiting, caching and user evaluation.

15 Code Walkthrough Plan

Session	Files	Learn	Task
1			
	README and start-studyforge.ps1		
		Service startup and ports	
			Run launcher and health check
2			
	frontend/app/page.tsx		
		Login, navigation, document selection and fetch	
			Trace generate function
3			
	main.py models and auth		
		Pydantic, signup, login and token	
			Explain current_user
4			
	main.py upload and retrieve		
		PDF validation, chunks, TF IDF and cosine	
			Draw data flow
5			
	main.py chat		
		Provider selection, streaming and fallback	
			Explain status, delta, done
6			
	main.py study tools		
		Question quality, fallback, submit and XP	
			Create a quiz
7			
	Components and CSS		
		Student experience and visual design	
			Prepare PPT screenshots

Method for Explaining Any Function

43. State input: what comes in?
44. State validation: what is checked?

45. State processing: which PDF, database, retrieval or AI action occurs?
46. State output: what returns or saves?
47. State failure: what happens if input or provider is invalid?

Example: generate_quiz receives document ID, count and logged-in user. It confirms ownership, prefers Gemini questions, uses local fallback if needed, stores complete questions in SQLite and returns only safe question data. It returns 422 if it cannot form distinct questions.

16 Limitations and Future Scope

Limitation	Reason	Future improvement
Scanned PDFs may lack text		
	Current upload does extraction, not full OCR	
		Add OCR such as Tesseract
TF IDF is word based		
	Different phrasing may not match	
		Use embeddings and vector database
SQLite is local scale		
	Production needs stronger concurrency and backup	
		Move to PostgreSQL
Custom signed token		
	Simple project implementation	
		Use standard sessions or JWT rotation
Provider dependency		
	Generation depends on quota and availability	
		Caching and multiple provider strategy
Fallback questions are literal		
	They use source sentences	
		Improve local NLP question generation

A strong viva answer includes both achievement and limitation. Explain what is implemented now, why it is appropriate for a BTech project and what you would change for deployment.

17 Final Revision Checklist

- I can explain the project in one minute.
- I can draw browser to backend to database to AI architecture.
- I can explain chunks, TF IDF and cosine similarity simply.
- I can explain password hash, token and ownership.
- I can show React calling a protected API.
- I can explain provider generation versus fallback.

- I can explain why quiz answers remain server-side.
- I can name validation and test checks.
- I can state limitations and future scope honestly.
- I have rehearsed opening, demo and closing.

Last Night Before Viva

48. Run START-STUDYFORGE.cmd and confirm both services.
49. Keep one short readable PDF ready.
50. Practise explanation without notes.
51. Keep this guide as speaker notes.
52. If AI is slow, explain retrieval and fallback instead of waiting silently.

Final Note

Your strongest viva performance comes from explaining what the project actually does. Start with the student journey, then explain technical choices. If asked about an advanced feature that is not present, say: In the current version we use TF IDF retrieval. For a more advanced deployment, I would evaluate embeddings and a vector database. This is honest and professional.